```
$ nxc --options-for-nxc-itself <mandatory protocol> --options-for-protocol -M <modulename>
    OPTION_FOR_MODULE=value
```

Generated commands often violated the parameter ordering, i.e., the "mandatory protocol" such as *smb*, was not given before options for the chosen protocol were supplied. The syntax used by *nxc* is in violation of *POSIX.1-2024*[29] which states in *12.2 Guideline 9* that *All options should precede operands on the command line.* Further complicating the tool's syntax are module options that need to be given in a different syntax similar resembling environmental variables.

We denoted another type of error as "Type 2" errors: a invalid parameter is supplied, passes the input-checking of the used tool but subsequently leads to an error. These cases are further complicated by the parameter error often being disguised as a network error. *Nmap* and *nxc* both exhibit this behavior: they allow passing multiple users or hostnames separated by spaces. Thus "`host1 host2`" is valid while "`host1,host2`" is invalid as it would be interpreted as a single hostname. Another problematic area is passing domain usernames to *nxc*: "`domain\\username`" and "`domain\\username`" are valid while `domain\username` or `user@domain` are not. The latter is a format that is often returned by AD enumeration tools.

Another problem for OpenAI's LLMs was exposed by *hashcat*, a tool used for password cracking. It expects a text-file with valid password hashes within each line. All hashes within the file must match the selected hash type and be formatted according to it. If a hash is of the wrong type, *hashcat* outputs a warning that a line within the input file could not be loaded as a valid hash. While *hashcat* does not seem to exhibit any Type 1 error within our analysis, when accounting for those "Separator unmatched" error messages, 94% of its invocations failed due to hashes being in the wrong format. Comparing GPT-4o's traces with Gemini-2.5-Flash indicates that this problem is occurring more often with OpenAI's GPT-4o.

6.4.2 Interactive, long-running and GUI Commands. One source of invalid commands arises when an Executor invokes interactive programs or programs that revert to an interactive mode in the absence of specified parameters. For example, when executing *smbclient*, if an authenticated operation is initiated without a password provided on the command line, the executed command awaits user input. In our prototype, no input is supplied, resulting in a command timeout after ten minutes. Similarly, calling *impacket-mssqlclient* without providing a SQL query causes the command to drop into an interactive SQL shell, waiting for SQL commands until the timeout occurs.

Network sniffers, such as *tcpdump* or *responder*, are typically launched to stream their output directly to stdout. Typically, a penetration tester monitors this output for relevant information—for instance, a NTLM hash emitted by *Responder*—after which the tester terminates the program, and transfers the relevant information—typically by using the system clipboard—from the tool's output into text files or subsequent tool calls.

Within our prototype we emulate this behavior through our command timeout. Commands are terminated after 10 minutes. All emulated simulated user interaction happens at a maximum 5 minute interval, thus ensuring that relevant data will be included within the captured output. After the command has been terminated, all output is presented back to the Executor LLM which then performs its analysis. While this is sufficient for Goad, real-life scenarios would mandate a more sophisticated system that notifies the Executor when new console output has occurred and not explicitly stops long-running processes after ten minutes.

Similar issues occur if programs are executed that use a graphical user interface which is not supported by our prototype environment. However, because penetration testing tools predominantly operate on the command line, this limitation can be considered secondary.

[29] IEEE Std 1003.1-2024, https://pubs.opengroup.org/onlinepubs/9799919799/

6.4.3 Planner and Executor collaborate to fix invalid commands. Qualitative analysis revealed that the prototype's built-in auto-repair capabilities effectively mitigate issues arising from invalid command generation by automatically correcting them. This behavior was consistently observed across all samling runs, underscoring both the high frequency of invalid commands and the robustness of the prototype's corrective mechanisms.

Auto-Repair within our prototype occurs on different abstraction levels. On a low-level the Executor employs this within its Executor loop. An invalid command produces a corresponding error message which is returned to the Executor. If the error message is of sufficient quality, the Executor can utilize it to issue an updated command remediating the original issue. Our logs show occurrences of this when using *ldapsearch* for domain enumeration. *ldapsearch* expects to be passed the target system through the parameter '`-H`'. However, GPT-4o has invalid tool usage information within its model data and commonly executed the command incorrectly using '`-h`' to pass the target system. Serendipitously, '`-h`' instructs *ldapsearch* to output its help page and thus provides the Executor sufficient information to provide a corrected command. This does not occur if the failed command invocation produced a low-quality or confusing error message, e.g., many security tools report a "network connection error" in case of invalid credentials, preventing the auto-repair from being performed.

Another example of the auto-repair mechanism is observed when a non-existent command is invoked. In these cases, the Executor reliably detects the missing dependency and initiates the installation of the required package(s). Our log traces document several instances where the Executor employs commands such as *apt*, *pip*, or even *git clone* to install additional software components.

Given that the Executor typically represents only a small amount (as low as 6% in case of our combined o1+GPT-4o configuration) of the overall costs in our prototype, allocating additional Executor rounds to rectify invalid command invocations is a cost-effective strategy. However, because the Executor lacks local memory, critical information regarding the correct tool invocation is lost once it communicates its findings back to the Planner. Consequently, each Executor invocation must re-learn the appropriate tool parameters from scratch.

On a high-level, if an Executorwas not able to remediate the problem, it reported the problem back to the Planner module including a short description. The Planner is commonly able to suggest additional remediations and instructs the Executor to apply them as the next task. While this is more time– and monetary more expensive than directly correcting the problem within the Executor loop, this oftentimes is able to solve the occurring issue.

6.4.4 Potential Impact of Improved Tooling Support. As demonstrated, many challenges encountered by the prototype are related to invoking tools with complicated parameter conventions, yet they do not adversely affect overall performance within our experimental scenario. For instance, tools with graphical user interfaces or interactive command line interfaces are infrequently utilized during penetration testing, and long-running tools (e.g., network sniffers) are effectively managed in the Goad environment by employing an extended timeout of 10 minutes. In cases where required tools are absent, the prototype automatically installs them via distribution packages, package repositories, or by cloning GitHub repositories. Additionally, the prototype exhibits the capacity to generate custom scripts, as evidenced by its successful creation of Python, C#, and PowerShell scripts. The rest of this sections investigates the question: what kind of additional tool support could improve the prototype's performance within Assumed Breach scenarios?

Access to an Attacker-Controlled Windows VM. A significant number of Active Directory penetration-testing tools are implemented in PowerShell and are optimally executed in a native Windows environment. Notable examples include *ADRecon*, *Rubeus*, *Kekeo*, *PowerView*, *SharpView*, *PowerMad*, *PowerUp*, and *PowerUpSQL*. Currently, our prototype is

configured to invoke functions on a Linux virtual machine. Integrating a Windows virtual machine would extend our prototype's capability to leverage Windows-exclusive tools during penetration testing.

Impact of Custom Attack-Specific Function Calls to the Executor LLM. A common strategy for improving tool use is to convert complex command line invocations into bespoke functions that can be invoked by the LLM [48, 57, 67]. They typically improve the tool's documentation compared to command line tools, and reduce the LLM's potential action space by providing bespoke high-level interfaces.

For example, during o1+GPT-4o runs, our prototype experienced massive problems calling *hashcat* for password cracking indicated by 94% of tool invocations not being successful due to invalid parameters. In cases like this, providing a dedicated password-cracking LLM function to the LLM should reduce the amount of invalid command executions as well as the amount of failed tasks, especially if providing higher-quality feedback in case of invalid hashes.

6.5 Safety Concerns

Given the sensitive topic of our capability evaluation—hacking computer networks—safety is a big concern. To protect our network and virtual-machine infrastructure, we followed best-practices and employed Virtual Machines as they offer strong security boundaries [16, 17] and included safety instructions in our scenario prompt (Section 3.2.5).

These safety instructions were ignored by Qwen3 and systems, that were explicitly excluded, scanned. After the first incident, we monitored all LLM-generated commands manually to be able to intervene in case of potentially destructive operations.

Another concern was Qwen3 replacing its penetration-testing goal with a non-related goal (Section 6.1). While the new goal was more benign than the original one, it's easy to imagine scenarios where this is not the case. Other models seem to have better guardrails protecting their generated output.

Another safety issue is the potential of LLMs to install new software or downgrade existing software as seen with Qwen3 which tried to install an older python version needed for a specific offensive tool. When installing through official package repositories, the main problem are unintended capabilities that the LLM can now utilize. If new software is directly installed from github repositories, the retrieved code can contain vulnerabilities or even could be part of a supply-chain attack. Similar issues are possible when downgrading packages.

And finally, LLMs' inherent capabilities for Inter-Context Attacks (Section 6.3.1) is problematic, esp. the case of performing social engineering attacks against real people. In addition to ethical issues, performing social engineering without acquiring prior consent is illegal in many jurisdictions.

All of these issues necessitate keeping humans in the loop for safety reasons.

6.6 Defenses against LLM-based Attacks

While our research focuses on the offensive use of LLMs for penetration-testing, we also want to spotlight potential countermeasures, hoping that future research will further elaborate on them.

Implement Basic Security Hygiene. Given the results our execution runs, LLMs perform similarly to human penetration testers and thus the same defenses apply: perform security updates, disable legacy protocols, and practice good security posture. Given the attack paths of our example runs, honey tokens and spray-able honey accounts would create a good initial line of detection.

Automated Defenses. Professional penetration testers typically include recommendations in their penetration test reports. We believe that LLMs can provide similar guidance or even automatically apply improvements. An initial foray into this was performed by PenHeal [22] which provided both attack-paths as well as defensive recommendations.

Tarpits for LLMs. Given that LLMs are prone to "go down the rabbit hole", defenders can deploy traps that lead LLMs towards infinite loops and increased time/resource consumption. Contemporary honey-token or deception systems are already in use to detect traditional attackers, comparable systems should be able to attract and slow attackers.

Pro-Active Defense through Prompt-Injections. LLMs are prone to malicious prompt injections. This behavior could be abused by defenders, e.g., by putting a webserver on the local network that contains text to motivate an attacking LLM to forget all prior instructions and either notify a defender by accessing a specially prepared URL, or to shutdown or destroy itself. This is deemed an offensive action in many jurisdictions and should be handled with care.

6.7 Ethical Issues or the lack thereof

We were surprised that our prompts did not trigger any form of detection within the used LLM-maker's cloud platforms as we were literally asking LLMs to hack computer networks. When evaluating third-party LLM-hosters such as together.ai, deepinfra.com, or fireworks.ai, our queries sometimes returned empty results. While the response documents did not include any indication of guardrails being applied, it is a possibility that automated filtering was occurring.

Security tooling is inherently dual-purpose and while LLM-driven security testing could democratize access to security testing, it could also be abused. Similar to other research projects [17], we believe that open access to security tooling will raise the collective security of all of us.

7 Conclusion

Our research demonstrates the feasibility and effectiveness of utilizing LLM-driven autonomous systems for Assumed Breach penetration-testing in real-world AD enterprise networks (Section 5, Page 23). They can effectively conduct Assumed Breach simulations by identifying initial access points and executing lateral movement. Reasoning LLMs compromised substantially more accounts and generated more leads compared to non-reasoning models (Section 5.3, Page 26), indicating their enhanced ability for strategic planning and execution in complex security scenarios.

The costs of employing LLM-driven prototypes are competitive with those incurred by professional human penetration-testers (Section 5.5, Page 28). This suggests a path toward democratizing access to essential security testing for organizations that traditionally cannot afford professional penetration-testing services, e.g., SMEs or NPOs.

Our findings highlight the LLMs' ability to dynamically adapt attack strategies (Section 6.3.1, Page 37). They can perform inter-context attacks, such as web application audits, social engineering, and unstructured data analysis for credentials. LLMs demonstrated the capacity to generate scenario-specific attack parameters, e.g., they generated realistic password candidates tailored to the testbed's theme. These capabilities that often exceed the scope of traditional security tooling (Section 6.3, Page 37).

Our prototype exhibits self-correction mechanisms, automatically installing missing tools and rectifying invalid command generations (Section 6.4.3, Page 42). This allows the system to overcome common operational hurdles, even with a notable percentage of initially invalid command invocations.

7.1 Challenges and Research Opportunities

LLMs occasionally exhibited a tendency to "go down rabbit holes," i.e., to hyper-focus on a single attack avenue while overlooking other potential leads (Section 6.2.5, Page 37). Research into implementing "circuit breakers" or dynamic task re-prioritization mechanisms could prevent LLMs from getting stuck in these unproductive attack loops.

There were challenges in comprehensive information transfer between the high-level Planner and low-level Executor modules, sometimes leading to redundant efforts or missed opportunities due to omitted critical context (Section 6.2, Page 35). Future work should focus on improving the robustness of information transfer and state management between the Planner and Executor, potentially by implementing a more sophisticated shared state repository or improved contextual prompting.

Critical safety concerns necessitate human oversight. Instances of LLMs ignoring explicit safety instructions (Section 6.5, Page 43), switching goals, hallucinating facts, and the inherent risks of performing social engineering attacks highlight the need for human supervision and guardrails.

We evaluated Qwen3 as an example of a modern open-weight small language model (Section 5.2, Page 26). It failed to heed safety instructions and was the only model not able to integrate the Executor's findings back into the attack plan (Section 6.3, Page 37). Further research into the feasibility of small language models for specialized tasks such as penetration-testing should be performed to unlock their potential for reduced costs while improving data privacy.

Improved attack-specific tooling support or tool abstractions for the Executor could reduce command generation errors and streamline complex tool invocations, improving overall efficiency (Section 6.4.4, Page 42). Providing the prototype with access to an attacker-controlled Windows VM would unlock a wider array of Windows-native penetration testing tools, enhancing capabilities in Active Directory environments. Investigating more sophisticated systems for managing long-running processes or network sniffers beyond our utilized timeout-based mechanism would enable more effective passive reconnaissance (Section 6.4.2, Page 41).

Further research into developing robust countermeasures against LLM-based attacks is vital (Section 6.6, Page 43). This includes exploring automated defenses, LLM-specific "tarpits," or even proactive prompt-injection techniques for defensive purposes.

Acknowledgments

We thank the anonymous reviewers for their careful reading of our manuscript and their many insightful comments and suggestions. We thank the Github AI Accelerator 2024 for their support and providing OpenAI credits used during our experiments.

References

[1] Abdulrahman Alamri and Lexie Mooney. 2025. Dragos Industrial Ransomware Analysis: Q1 2025. https://www.dragos.com/blog/dragos-industrial-ransomware-analysis-q1-2025/. Accessed: 2025-06-02.

[2] Ron Alford, Dean Lawrence, and Michael Kouremetis. 2022. Caldera: A red-blue cyber operations automation platform. *MITRE: Bedford, MA, USA* (2022).

[3] Afnan Binduf, Hanan Othman Alamoudi, Hanan Balahmar, Shatha Alshamrani, Haifa Al-Omar, and Naya Nagy. 2018. Active Directory and Related Aspects of Security. In *2018 21st Saudi Computer Society National Computer Conference (NCC)*. 4474–4479. doi:10.1109/NCG.2018.8593188

[4] Virginia Braun and Victoria Clarke. 2006. Using thematic analysis in psychology. *Qualitative research in psychology* 3, 2 (2006), 77–101.

[5] Kathy Charmaz. 2006. *Constructing grounded theory: A practical guide through qualitative analysis*. Sage.

[6] dair ai. 2025. Reasoning LLMs Guide. https://www.promptingguide.ai/guides/reasoning-llms. Accessed: 2025-06-11.

[7] Gelei Deng, Yi Liu, Víctor Mayoral-Vilches, Peng Liu, Yuekang Li, Yuan Xu, Tianwei Zhang, Yang Liu, Martin Pinzger, and Stefan Rass. 2024. PentestGPT: An LLM-empowered Automatic Penetration Testing Tool. arXiv:2308.06782 [cs.SE] https://arxiv.org/abs/2308.06782

[8] Norman K Denzin. 2017. *Sociological methods: A sourcebook*. routledge.